Diagrama de Classes UML

Lendo as setas, as cardinalidades e transformando o diagrama em código Java

Sistema acadêmico: Secretaria, Pessoa, Aluno, Professor, Disciplina, Turma, Matrícula, Diário de Classe

Roteiro da aula

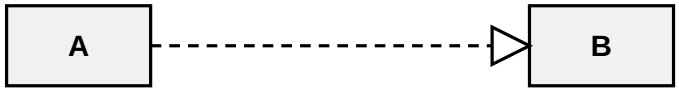
- 1. O diagrama completo e como lê-lo
- 2. Legenda das setas (realização, herança, agregação, composição, associação, dependência)
- 3. Cardinalidades: 1, 0..*, 1..*, 0..1
- 4. Cada relação do nosso diagrama, explicada em cima da imagem
- 5. Conceitos de Java usados: interface, @Override, polimorfismo, sobrecarga
- 6. Tabela UML para Java e roteiro de codificação passo a passo

1. O diagrama completo



Como ler: cada caixa é uma classe (topo: nome, meio: atributos, base: métodos). **I** = interface, **A** = classe abstrata, **C** = classe concreta. O sinal **-** é *private* e o **+** é *public*. As linhas entre as caixas são as relações, e o desenho na ponta da linha diz o **tipo** da relação.

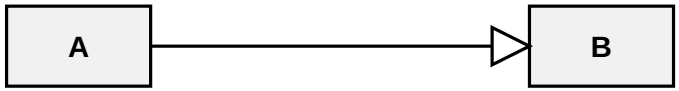
2. Legenda das setas (A e B são duas classes quaisquer)



Realização (implementação)

Linha **tracejada** com triângulo **vazado** apontando para a interface. Lê-se: A implementa B.

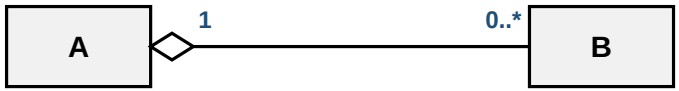
Em Java: `class A implements B`
No seu diagrama: Secretaria implementa Matriculavel. DiarioDeClasse implementa Avaliacao.



Herança (generalização)

Linha **contínua** com triângulo **vazado** apontando para a classe mãe. Lê-se: A é um B.

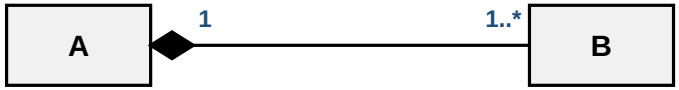
Em Java: `class A extends B`
No seu diagrama: Aluno e Professor herdam de Pessoa (abstrata).



Agregação

Linha contínua com **losango vazado** no lado do "todo". Lê-se: A tem B, mas B existe sem A.

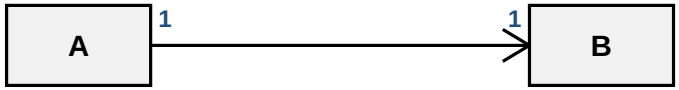
Em Java: A tem um `List<B>` como atributo; os B são criados fora e adicionados.
No seu diagrama: Secretaria tem Matriculas; DiarioDeClasse tem Notas e Frequencias.



Composição (não aparece no seu diagrama)

Linha contínua com **losango preenchido** no lado do todo. A parte **morre junto** com o todo.

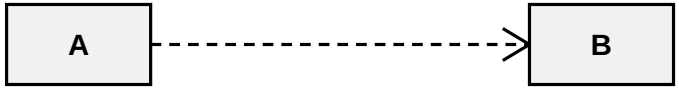
Em Java: A cria os B dentro de si (`new B()` no construtor) e ninguém mais os usa.
No seu diagrama: Não há composição. Se uma Nota só existisse dentro do DiarioDeClasse e sumisse com ele, seria composição.



Associação (simples, com ou sem seta)

Linha contínua sem enfeite: A conhece B. Com uma seta aberta em V, indica quem enxerga quem (navegabilidade).

Em Java: A tem um atributo do tipo B (`private B b;`).
No seu diagrama: Matricula liga Aluno e Turma; Turma e Disciplina (muitos para muitos).



Dependência

Linha **tracejada** com seta aberta em V. A apenas **usa** B de passagem (parâmetro, variável local).

Em Java: B aparece só como parâmetro ou variável local de um método de A.
No seu diagrama: Não aparece (mas matricular(aluno, turma) usa Aluno e Turma como parâmetros).

3. Cardinalidades (multiplicidade)

Os números nas pontas das linhas dizem **quantos objetos** daquele lado participam da relação. A leitura é sempre: *para 1 objeto do lado A, quantos objetos do lado B?*

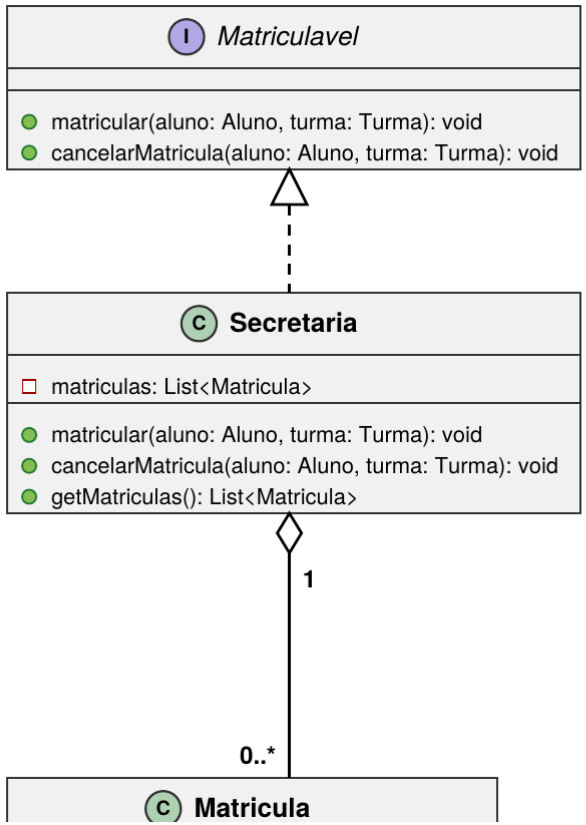
Notação	Significa	Exemplo	Em Java
1	Exatamente um (obrigatório)	Uma Matricula tem exatamente 1 Aluno	`private Aluno aluno;`
0..1	Zero ou um (opcional)	Uma Turma pode ter 0 ou 1 monitor	`private Monitor monitor; // pode ser null`
0..*	Zero ou muitos	Uma Secretaria tem 0 ou muitas Matriculas	`private List<Matricula> matriculas;`
1..*	Um ou muitos (pelo menos um)	Uma Turma cursa ao menos 1 Disciplina	`List<Disciplina>` + regra: não aceitar lista vazia
3..5	Faixa fixa	Uma equipe tem de 3 a 5 membros	`List` + validação de tamanho

Truque de leitura

- Olhe o número que está **na ponta oposta**: ele diz quantos desse lado cada objeto do outro lado possui.
- Cardinalidade **1** ou **0..1** vira **um atributo simples**. Cardinalidade **0..*** ou **1..*** vira **uma List**.
- No losango (agregação), o **1** fica junto do losango: 1 Secretaria tem 0..* Matriculas.

4. Relações do nosso diagrama

4.1 Realização: Secretaria implementa Matriculavel



A seta **tracejada com triângulo vazado** sai de Secretaria e aponta para a interface Matriculavel.

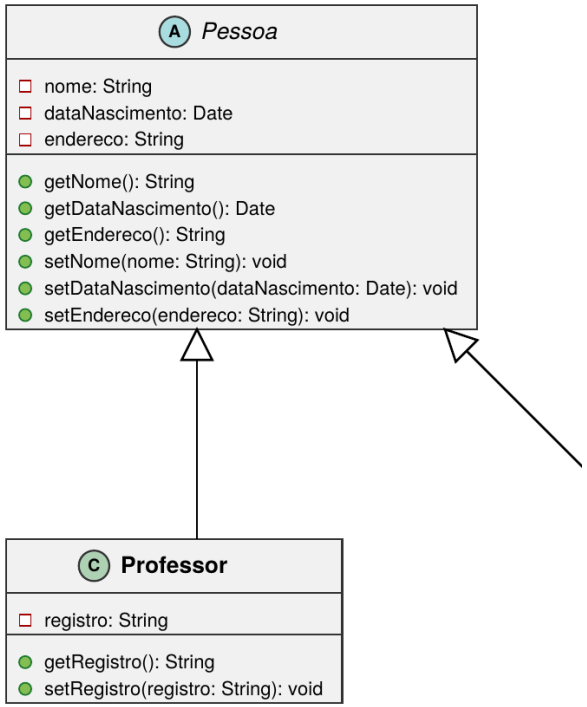
A interface só declara **o que** fazer (`matricular` e `cancelarMatricula`). A Secretaria diz **como** fazer, repetindo os dois métodos com `@Override`.

O mesmo desenho aparece entre **DiarioDeClasse** e **Avaliacao**.

```
public interface Matriculavel {
    void matricular(Aluno aluno, Turma turma);
    void cancelarMatricula(Aluno aluno, Turma turma);
}

public class Secretaria implements Matriculavel {
    @Override
    public void matricular(Aluno aluno, Turma turma) {
        // cria a Matricula e guarda na lista
    }
    @Override
    public void cancelarMatricula(Aluno aluno, Turma turma) {
        // remove a Matricula da lista
    }
}
```

4.2 Herança: Aluno e Professor herdam de Pessoa



Linhas **contínuas** com triângulo **vazado** apontando para **Pessoa** (a classe mãe, marcada com **A** de abstrata).

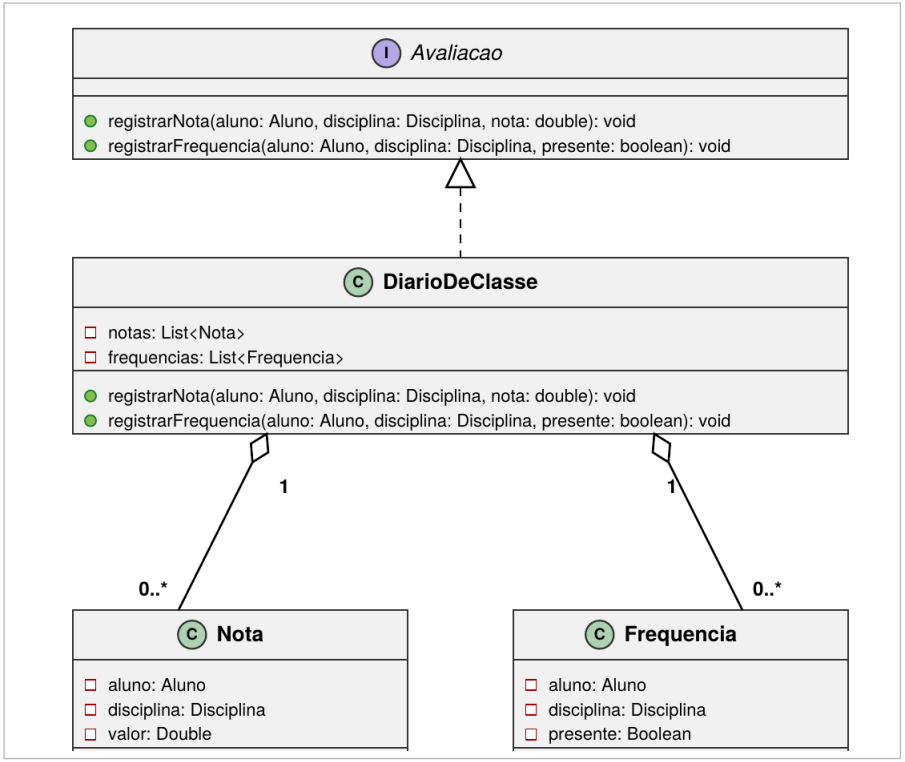
Tudo que está em Pessoa (nome, dataNascimento, endereco e seus getters e setters) é herdado por Aluno e Professor, que acrescentam apenas o que é só deles: **matricula** (Aluno) e **registro** (Professor).

Por ser **abstrata**, não dá para fazer `new Pessoa()`, apenas `new Aluno()` ou `new Professor()`.

```
public abstract class Pessoa { ... }
public class Aluno extends Pessoa { ... }
public class Professor extends Pessoa { ... }

Pessoa p = new Aluno();   // polimorfismo
```

4.3 Agregação: o losango vazado (3 ocorrências)



Zoom em **DiarioDeClasse**, **Nota** e **Frequencia**. O losango fica no Diário (o todo), com **1**. Em Nota e Frequencia está o **0..***. A Secretaria com suas Matriculas segue o mesmo desenho.

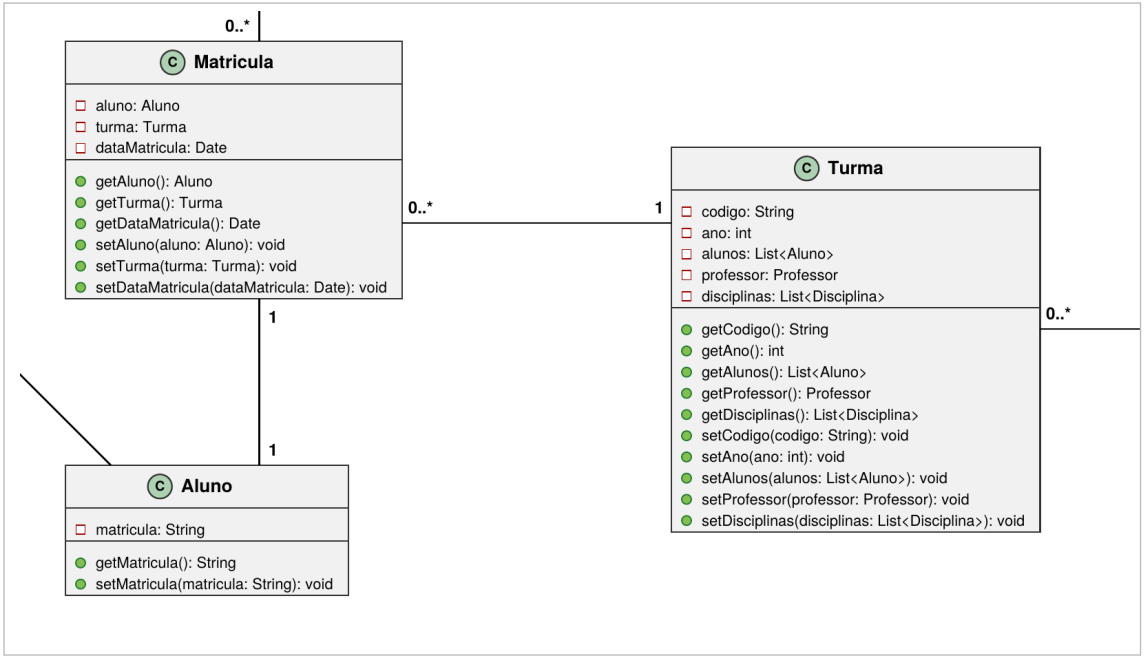
Por que agregação e não composição? O losango está **vazio**: a parte tem vida própria. Uma Matricula ou uma Nota existem como objetos independentes e só são *reunidos* pelo todo.

Regra de ouro para codificar: o todo tem uma `List` inicializada no construtor e um método para adicionar partes. Quem cria as partes é quem está **fora**.

```
public class DiarioDeClasse implements Avaliacao {
    private List<Nota> notas = new ArrayList<>();
    private List<Frequencia> frequencias = new ArrayList<>();
    // registrarNota cria a Nota e faz notas.add(...)
}
```

Todo (lado do losango)	Cardinalidade	Parte	Leitura	Atributo em Java
Secretaria	1 para 0..*	Matricula	Uma Secretaria guarda muitas Matriculas	`List<Matricula> matriculas`
DiarioDeClasse	1 para 0..*	Nota	Um Diário guarda muitas Notas	`List<Nota> notas`
DiarioDeClasse	1 para 0..*	Frequencia	Um Diário guarda muitas Frequências	`List<Frequencia> frequencias`

4.4 Associação simples: Matricula liga Aluno e Turma



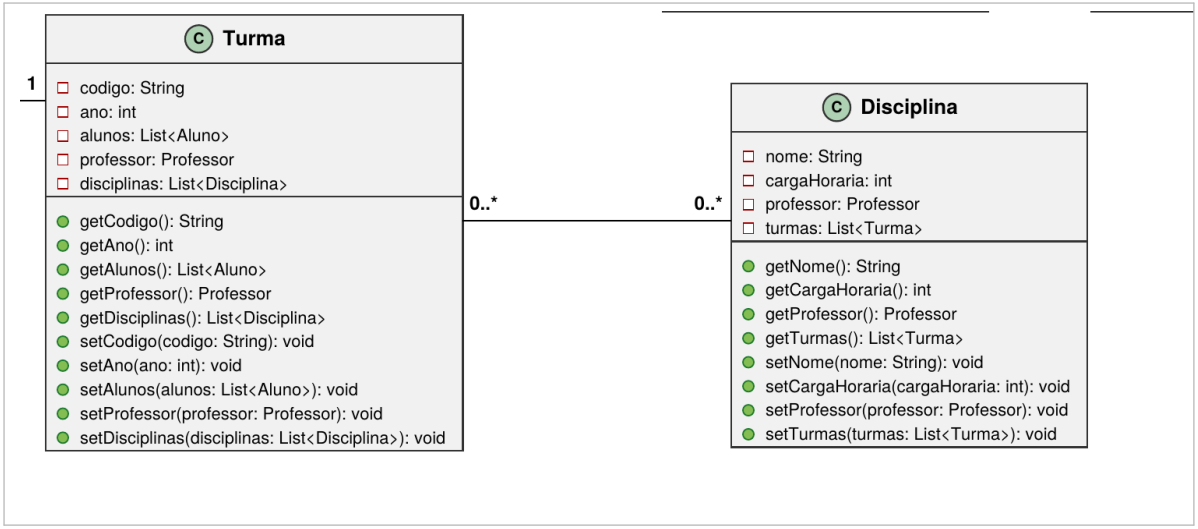
As linhas **sem enfeite** (sem losango, sem triângulo, sem tracejado) são **associações**: a Matricula **conhece** um Aluno e uma Turma.

Como a Matricula é quem aponta, ela guarda **um atributo de cada**:

```
public class Matricula {
    private Aluno aluno;
    private Turma turma;
    private Date dataMatricula;
}
```

Como ler os números: aqui Turma é a classe de alunos (ex: 2º DSM). Cada Aluno está em **uma** Turma e tem **uma** Matrícula (os dois **1** do lado do Aluno). Já uma Turma tem **muitos** alunos, logo **0..*** Matrículas.

4.5 Associação muitos para muitos: Turma e Disciplina



Uma Turma (2º DSM) cursa **várias** disciplinas e uma Disciplina é dada a **várias** Turmas. Por isso os dois lados são **0..***. Em Java, **cada lado guarda uma lista do outro**:

```
public class Turma {
    private List<Disciplina> disciplinas = new ArrayList<>();
}

public class Disciplina {
    private List<Turma> turmas = new ArrayList<>();
}
```

4.6 Resumo: todas as relações do diagrama

Origem	Tipo	Destino	Cardinalidade	Código Java
Secretaria	Realização	Matriculavel	-	`implements Matriculavel`
DiarioDeClasse	Realização	Avaliacao	-	`implements Avaliacao`
Aluno	Herança	Pessoa	-	`extends Pessoa`
Professor	Herança	Pessoa	-	`extends Pessoa`
Secretaria	Agregação	Matricula	1 para 0..*	`List<Matricula>`
DiarioDeClasse	Agregação	Nota	1 para 0..*	`List<Nota>`
DiarioDeClasse	Agregação	Frequencia	1 para 0..*	`List<Frequencia>`
Turma	Associação	Disciplina	0..* e 0..*	`List<Disciplina>` e `List<Turma>`
Matricula	Associação	Aluno	1 e 1	`private Aluno aluno;`
Matricula	Associação	Turma	0..* e 1	`private Turma turma;`

Composição e dependência: **nenhuma** no diagrama.

5. Conceitos de Java que o diagrama usa

Interface

- Declara métodos **abstratos** (sem corpo). Quem implementa é obrigado a escrever todos.
- **Não** tem atributos de instância. Só constantes, que são sempre `public static final`.
- Pode ter métodos `default`, `static` e `private` (Java 9+).
- Uma classe pode implementar **várias** interfaces, mas herdar de **uma** classe só.

@Override

- Vale tanto para sobrescrever método da classe mãe quanto para implementar método da interface.
- Não é obrigatório, mas é **boa prática**: o compilador avisa se você errou o nome ou os parâmetros.

Polimorfismo

```
List<Pessoa> pessoas = new ArrayList<>();
pessoas.add(new Aluno());
pessoas.add(new Professor());

for (Pessoa p : pessoas) {
    p.exibirDados();   // sem instanceof!
}
```

A JVM escolhe em **tempo de execução** qual `exibirDados()` chamar, conforme o objeto real. Só é preciso *cast* para chamar algo exclusivo do subtipo (ex: `getRegistro()`).

Sobrecarga (overload) x Sobrescrita (override)

	Sobrecarga (overload)	Sobrescrita (override)
Assinatura	Mesmo nome, parâmetros **diferentes**	Mesmo nome, mesmos parâmetros
Onde	Na mesma classe ou entre mãe e filha	Entre mãe e filha (ou interface e classe)
Decisão	Em tempo de **compilação**	Em tempo de **execução**
Anotação	Não existe @Overload	`@Override` (recomendada)

6. Tabela de tradução: UML para Java

No diagrama	No código Java
Caixa com **I** (interface)	`public interface Nome { ... }`
Caixa com **A** (abstrata)	`public abstract class Nome { ... }`
Caixa com **C**	`public class Nome { ... }`
- nome: String	`private String nome;`
+ getNome(): String	`public String getNome() { return nome; }`
+ setNome(nome: String): void	`public void setNome(String nome) { this.nome = nome; }`
Seta tracejada + triângulo vazado	`class A implements B`
Seta contínua + triângulo vazado	`class A extends B`
Losango vazado, 0..*	`private List<B> bs = new ArrayList<>();`
Linha simples, cardinalidade 1	`private B b;`
Tipo Date, List	`import java.util.Date; import java.util.List; import java.util.ArrayList;`

7. Roteiro para codificar com a turma

Ordem sugerida: das classes que **não dependem de ninguém** para as que dependem. Assim o código compila a cada etapa.

Passo	Classes	O que ensinar	Teste rápido
1	Pessoa (abstrata), Aluno, Professor	Herança, abstract, super, getters e setters	Criar um Aluno e um Professor e imprimir o nome
2	Matriculavel e Avaliacao (interfaces)	Interface só com métodos	Compilar
3	Disciplina e Turma	Associação muitos para muitos: uma List em cada lado	Criar 2 disciplinas e 1 turma, ligar nos dois lados
4	Matricula	Associação com Aluno e Turma	Criar uma Matricula com data
5	Secretaria	implements, @Override, lista de Matricula	matricular() e conferir getMatriculas().size()
6	Nota, Frequencia, DiarioDeClasse	implements Avaliacao, agregação dupla	Registrar nota e frequência de um aluno
7	Main	Montar tudo e mostrar o polimorfismo	Rodar o fluxo completo

Código-base dos passos 1, 2 e 5 está na próxima página.

Esqueletos de código

```
import java.util.Date;

public abstract class Pessoa {
    private String nome;
    private Date dataNascimento;
    private String endereco;

    public String getNome() { return nome; }
    public void setNome(String nome) { this.nome = nome; }
    // ... demais getters e setters
}

public class Aluno extends Pessoa {
    private String matricula;
    public String getMatricula() { return matricula; }
    public void setMatricula(String m) { this.matricula = m; }
}

public class Professor extends Pessoa {
    private String registro;
    public String getRegistro() { return registro; }
    public void setRegistro(String r) { this.registro = r; }
}
```

```
public class Matricula {
    private Aluno aluno;
    private Turma turma;
    private Date dataMatricula;
    // getters e setters
}

public class Secretaria implements Matriculavel {
    private List<Matricula> matriculas = new ArrayList<>();

    @Override
    public void matricular(Aluno aluno, Turma turma) {
        Matricula m = new Matricula();
        m.setAluno(aluno);
        m.setTurma(turma);
        m.setDataMatricula(new Date());
        matriculas.add(m);
    }

    @Override
    public void cancelarMatricula(Aluno aluno, Turma turma) {
        matriculas.removeIf(m -> m.getAluno() == aluno
                              && m.getTurma() == turma);
    }

    public List<Matricula> getMatriculas() { return matriculas; }
}
```